"A New Golden Age for Computer Architecture"

title of 2019 paper written by authors of our textbook (and also the title of their 2018 Turing award lecture ...)

rapid innovation in computer architecture continues!

areas of particular current interest include:

specialized accelerators / domain-specific architectures ("The next decade will see a Cambrian explosion of novel computer architectures")

- e.g. for machine learning applications (Google TPUv7, Amazon Trainium3, Microsoft Maia 200, …)

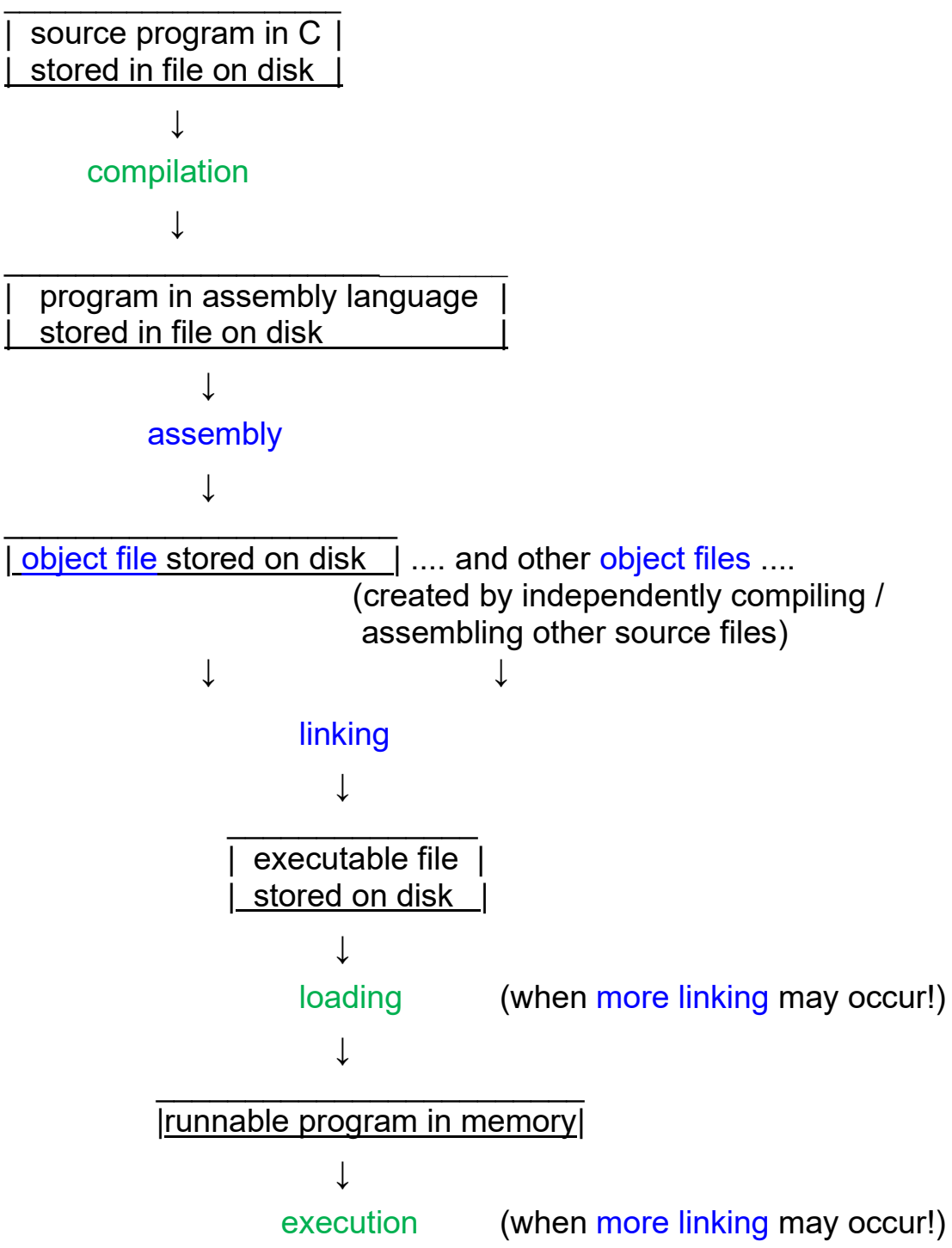
security

including how to best address some troubling types of security vulnerabilities ("Spectre", "Meltdown")

Program assembly and linking

(reference: text Section 2.12)

what happens when a C program is compiled & run?



outputs from assembly / inputs to linking are object files ... what do these contain?

in addition to code and data, object files must contain other information needed for linking

- symbol table information, including external label information, about names defined in the module that are usable in other modules, and external reference information, about uses of names defined in other modules
- relocation information, about machine language instructions with fields that depend on addresses program occupies

```
add s1, s2, s3   ⇨ no

jal ra, P    #“P” in different module ⇨ yes

bne s1, s2, loop    #“loop” in same
                                module ⇨ no

la s0, A    # “A” in data section ⇨ yes
```

program assembly

to translate instructions like "bne s1, s2, loop" and "j loop", need to know the memory address that the name "loop" represents

assembler uses an assumed starting address for the code, and an assumed starting address for the data; could for example use address 0 for both

- addresses will be "fixed" later!

because of forward references, won't work to do just a single pass through source program

two pass assembly

in first pass, create symbol table

- keep track of where in memory each machine language instruction or data value would be placed (given assumed starting location); when label encountered, enter in table together with corresponding address

in second pass, create machine language code

linking

if broadly define "linking" as the process of combining together multiple, independently developed program units, can be done at:

- coding time (manually)
- compilation/assembly time (using "include" facility)
- after compilation/assembly, but before loading & execution (using static linker)
- load time or during execution (using dynamic linker)

linking after compilation/assembly, but before execution ("static linking")

linker needs to concatenate the code segments of the object files being linked, and concatenate their data segments

but how do we know where to place the concatenated segments in the memory address space? (typically we have no idea where in the system main memory the program will actually be loaded and run!)

use convention for starting locations in memory space (e.g., see Figure 2.13 in text for a RISC-V software convention for 64-bit systems); addresses are "fixed" as program executes!

typically:

- stack starts at a high address and grows down
- text segment (code) is allocated at a low memory address, above it is allocated the static data (the rodata, data, and bss segments)
- heap starts above the static data and grows up

advantages of placing stack and heap at opposite ends?

in our environment, what address does the application program code start at? how about the allocated stack?

linker operation

- since modules moved in address space, need to add relocation constant to the addresses recorded for external labels
- use (fixed) external addresses and relocation information to patch addressing fields

suppose for example have two object files A and B

object file A:

- 300_{16} bytes of machine code
- calls two procedures defined in object file B, proc1 and proc2, and so needs to include external reference information (assembler might assume that any use of a name that is not defined by use as a label is an external reference)

object file B:

- 250_{16} bytes of machine code for procedures proc1 and proc2
- needs to include external label information (in our environment, would inform the assembler that proc1 and proc2 should be visible externally using ".globl")

if machine code for A is put first starting at address 400000_{16}, what would be the relocation constant for object file B code?

dynamic linking

used with DLLs ("dynamic linked libraries") containing library routines

two variants:

- "pre-execution" ("load-time"): dynamic linking done during program start-up
- "on-demand" ("lazy"): a routine only linked if actually called, during execution

on-demand implementation uses "dummy routines" that initially do an *indirect jump* (i.e., a jump to an address given in a memory word) to code that invokes the dynamic linker

when a routine is first called, the dynamic linker includes that routine in the application address space, and changes the address given in the memory word to the address of that routine

subsequent calls to the "dummy routine" will then do an indirect jump to the desired routine rather than to the code invoking the dynamic linker

advantages/disadvantages compared to static linking?